

DATA SCIENCE LEARNING NOTES

DAY 05 • PYTHON LISTS AND ADVANCED LIST OPERATIONS

Student: Muhammad Ahmad

Goal: Data Scientist

Journey: 60-Day Intensive Data Science Learning

Day 5 Overview

Lists store multiple values in one variable. They are an important Python foundation for data processing before NumPy arrays and Pandas DataFrames.

Learning Flow

lists → indexing → negative indexing → slicing → updating → adding → removing → sorting → searching
→ loops → nested lists → list comprehension → filtering

1. List Creation

A list stores multiple values.

```
marks = [85, 72, 90, 65, 78]
print(marks)
```

2. Indexing

Python indexing starts from 0.

```
marks = [85, 72, 90, 65]
print(marks[0])
print(marks[2])
```

3. Negative Indexing

Negative indexes start from the last element.

```
marks = [85, 72, 90, 65]
print(marks[-1])
print(marks[-2])
```

4. len()

Returns the number of elements.

```
marks = [85, 72, 90, 65]
print(len(marks))
```

5. Updating

Lists are mutable, so values can change.

```
marks = [85, 72, 90]
marks[1] = 80
print(marks)
```

6. append()

Adds one element at the end.

```
marks = [70, 80, 90]
marks.append(95)
print(marks)
```

7. insert()

Adds an element at a specific position.

```
marks = [70, 80, 90]
marks.insert(1, 75)
print(marks)
```

8. extend()

Adds multiple elements.

```
marks = [70, 80, 90]
marks.extend([60, 85, 95])
print(marks)
```

9. remove()

Removes the first matching value.

```
marks = [70, 80, 90, 80]
marks.remove(80)
print(marks)
```

10. pop()

Removes and returns an element.

```
marks = [70, 80, 90]
removed = marks.pop()
print(removed)
print(marks)
```

11. clear()

Removes all elements.

```
marks = [70, 80, 90]
marks.clear()
print(marks)
```

12. Searching

Use in to check whether a value exists.

```
courses = ["Python", "SQL", "Statistics"]
print("Python" in courses)
print("Java" in courses)
```

13. Slicing

Extracts part of a list. The stop index is excluded.

```
marks = [10, 20, 30, 40, 50]
print(marks[1:4])
print(marks[:3])
print(marks[2:])
```

14. Reverse with Slicing

[::-1] returns the list in reverse order.

```
numbers = [1, 2, 3, 4, 5]
print(numbers[::-1])
```

15. sort()

sort() changes the original list.

```
marks = [70, 45, 90, 60, 85]
marks.sort()
print(marks)
marks.sort(reverse=True)
print(marks)
```

16. sorted()

Returns a sorted copy and keeps the original list unchanged.

```
marks = [70, 45, 90, 60]
new_marks = sorted(marks)
print(marks)
print(new_marks)
```

17. count() and index()

count() finds frequency. index() finds the first position.

```
marks = [80, 70, 80, 90, 80]
print(marks.count(80))
print(marks.index(90))
```

18. sum(), min(), max()

Useful built-in functions for basic numerical analysis.

```
marks = [70, 80, 90, 65]
print(sum(marks))
print(min(marks))
print(max(marks))
```

19. Loop Through a List

Loops process each element.

```
marks = [70, 80, 90, 65]
for mark in marks:
    print(mark)
```

20. Filter with a Loop

Create a new list containing passing marks.

```
marks = [45, 78, 90, 32, 65]
passed = []
for mark in marks:
    if mark >= 50:
        passed.append(mark)
print(passed)
```

21. List Comprehension

A concise way to create a new list.

```
marks = [45, 78, 90, 32, 65]
passed = [mark for mark in marks if mark >= 50]
print(passed)
```

22. Transform Values

List comprehension can transform values.

```
numbers = [10, 20, 30, 40]
squares = [number ** 2 for number in numbers]
print(squares)
```

23. Nested Lists

A list can contain other lists.

```
students = [["Ahmad", 85], ["Ali", 72], ["Usman", 65]]
print(students[0])
print(students[0][0])
print(students[0][1])
```

24. List of Dictionaries

This resembles structured records you will later process with Pandas.

```
students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
    {"name": "Usman", "marks": 65}
]
for student in students:
    print(student["name"], student["marks"])
```

25. Filter Student Records

Filter records using a condition.

```
students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
]
```

```
{"name": "Usman", "marks": 45}  
]  
passed = [s for s in students if s["marks"] >= 50]  
print(passed)
```

26. Practical Data Processing

Combine filtering and basic statistics.

```
marks = [45, 78, 90, 32, 65, 88, 95]  
passed = [m for m in marks if m >= 50]  
failed = [m for m in marks if m < 50]  
print("Passed:", passed)  
print("Failed:", failed)  
print("Average:", sum(marks) / len(marks))  
print("Highest:", max(marks))  
print("Lowest:", min(marks))
```

DAY 5 PRACTICE

1. Create a list of 10 numbers and find their sum.
2. Find the largest and smallest number without max() and min().
3. Separate Pass and Fail marks into two lists.
4. Reverse a list using slicing.
5. Remove duplicate values from a list.
6. Create a list comprehension for even numbers.
7. Create a list comprehension that squares every number.
8. Filter student dictionaries where marks are ≥ 70 .

DAY 5 MINI PROJECT

Build a Student Marks Analyzer. Calculate total students, average marks, highest marks, lowest marks, passing students, failing students, and passing and failing percentages.

```
students = [
{"name": "Ahmad", "marks": 85},
{"name": "Ali", "marks": 72},
{"name": "Usman", "marks": 45},
{"name": "Hamza", "marks": 91},
{"name": "Bilal", "marks": 58}
]

marks = [student["marks"] for student in students]
passed = [m for m in marks if m >= 50]
failed = [m for m in marks if m < 50]

total = len(students)
average = sum(marks) / total
pass_percentage = len(passed) / total * 100
fail_percentage = len(failed) / total * 100

print("Total Students:", total)
print("Average:", round(average, 2))
print("Highest:", max(marks))
print("Lowest:", min(marks))
print("Passing:", len(passed))
print("Failing:", len(failed))
print("Pass %:", round(pass_percentage, 2))
print("Fail %:", round(fail_percentage, 2))
```

QUICK REVISION

```
indexing → access an element
slicing → extract part of a list
append() → add one element
insert() → add at a position
extend() → add multiple elements
remove() → remove a value
pop() → remove and return
sort() → sort original list
sorted() → return sorted copy
count() → count a value
index() → find first position
sum() → total
min()/max() → smallest/largest
list comprehension → concise list creation
```

DATA SCIENCE CONNECTION

Lists teach collections, filtering, transformation, and aggregation. These concepts prepare you for NumPy arrays and Pandas DataFrames.

DAY 5 COMPLETE

Next: Day 6, Python Tuples, Sets, and Dictionaries.